

Módulo 9 — Projeto CRUD Completo + Tratamento de Erros

Jornada DEV START · Aula 9 · 29/07 (quarta) · Programa START — TOTVS Paulista
Material entregue ao final da Aula 9. Guarde para consultar sempre que precisar.

Onde você está na jornada

1 → 2 → 3 → 4 → 5 → 6 | 7 → 8 → [9] → 10
Fundamentos (Harbour) | Protheus (ADVPL)
▲ você está aqui

Na Aula 8 você aprendeu a criar **um** cadastro com **AxCadastro** e **mBrowse**. Agora chegou a hora de juntar tudo num **projeto de verdade**: um mini-sistema com **duas tabelas relacionadas**, uma **biblioteca de funções**, **validações cruzadas** e **menus**. E, enquanto você constrói esse sistema, vai descobrir o que separa um código de aula de um código de produção: **o tratamento de erros**. Este é o módulo em que você deixa de “fazer uma telinha” e passa a **construir software profissional**. 🛠️

Roteiro da aula (2h)

Bloco	Tempo	O quê
Retomada	~10 min	Revisão do CRUD da Aula 8 + correção de um exercício-chave
Conteúdo	~60 min	Projeto de 2 tabelas (SZ1/SZ2), lib, validações cruzadas e tratamento de erros
Mão na massa	~40 min	Codar juntos a rotina de salvar “à prova de falhas”
Q&A / networking	~10 min	Dúvidas abertas e integração da turma

🎯 Objetivos do módulo

Ao final desta aula você será capaz de: - Modelar e configurar **duas tabelas relacionadas** (1:N) no dicionário de dados - Criar uma **biblioteca de funções comuns** (lib) para não repetir código - Ligar as tabelas com **campos virtuais**, **gatilhos** e um **botão de navegação** - Implementar **validações cruzadas** e proteger a **integridade referencial** - Capturar erros com **BEGIN SEQUENCE / RECOVER / END SEQUENCE** - Usar **transações** (**BeginTran/CommitTran/RollBackTran**) e **logs** para operações seguras

9.1 O projeto: um mini-sistema de verdade

Você vai construir um sistema de **Controle de Contatos e Histórico de Interações** — algo que qualquer equipe comercial usaria no dia a dia:

- **SZ1 — Contatos:** registra os contatos de clientes (cliente, assunto, data)
- **SZ2 — Interações:** detalha cada interação feita com aquele contato (ligação, e-mail, visita...)

Cada contato (SZ1) pode ter **várias** interações (SZ2). É uma relação de **um para muitos (1:N)** — a mesma estrutura que aparece em pedido→itens, cliente→endereço, nota→parcelas. Aprender a construir **um** relacionamento 1:N ensina você a construir **todos**.

SZ1 – Contatos (1)

Z1_CODIGO (PK)
Z1_CLIENTE
Z1_NOME (Virtual)
Z1_ASSUNTO
Z1_DATA / Z1_HORA

—1:N—

SZ2 – Interações (N)

Z2_CONTAT (FK → SZ1)
Z2_SEQUEN
Z2_TIPO
Z2_DESCRI
Z2_DATA / Z2_HORA
Z2_USUAR

Relacionamento 1:N entre Contatos e Interações

💡 **Regra de ouro do relacionamento:** a tabela “filha” (SZ2) sempre guarda a **chave da mãe** (o campo **Z2_CONTAT** aponta para **Z1_CODIGO**). É por esse campo que ligamos as duas.

9.2 Modelando os dados (dicionário)

Todo sistema profissional começa pela **modelagem**. Antes de uma linha de rotina, as tabelas precisam existir no dicionário de dados (o mesmo que você conheceu na Aula 7).

Estrutura da SZ1 — Contatos

Título	Campo	Tipo	Tam	Dec	Contexto
Filial	Z1_FILIAL	C	2	0	Real
Código	Z1_CODIGO	C	6	0	Real
Cliente	Z1_CLIENTE	C	6	0	Real
Loja Cliente	Z1_LOJACLI	C	2	0	Real
Nome Cliente	Z1_NOME	C	40	0	Virtual
Assunto	Z1_ASSUNTO	C	60	0	Real
Data	Z1_DATA	D	8	0	Real
Hora	Z1_HORA	C	5	0	Real

Estrutura da SZ2 — Interações

Título	Campo	Tipo	Tam	Dec	Contexto
Filial	Z2_FILIAL	C	2	0	Real
Contato (→SZ1)	Z2_CONTAT	C	6	0	Real
Seq.	Z2_SEQUEN	C	3	0	Real
Tipo	Z2_TIPO	C	1	0	Real
Descrição	Z2_DESCRI	C	100	0	Real
Data	Z2_DATA	D	8	0	Real
Hora	Z2_HORA	C	5	0	Real
Usuário	Z2_USUAR	C	20	0	Real
Cód. Contato	Z2_CODIGO	C	6	0	Virtual
Assunto Cont.	Z2_ASSUNT	C	60	0	Virtual

Registrar as tabelas (SX2) e os índices (SIX)

SX2 — tabelas:

Prefixo	Nome	Modo
SZ1	Contatos	C (Compartilhado)
SZ2	Interações	C (Compartilhado)

SIX — índices da SZ1:

Ordem	Expressão	Descrição
1	Z1_FILIAL + Z1_CODIGO	Chave primária
2	Z1_FILIAL + Z1_CLIENTE + Z1_LOJACLI	Por cliente
3	Z1_FILIAL + DTOS(Z1_DATA)	Por data

SIX — índices da SZ2:

Ordem	Expressão	Descrição
1	Z2_FILIAL + Z2_CONTAT + Z2_SEQUEN	Chave primária
2	Z2_FILIAL + DTOS(Z2_DATA)	Por data

Domínio dos tipos de interação (SX5)

O campo Z2_TIP0 guarda um código de uma letra. Para o usuário ver o nome por extenso, cadastre uma tabela de domínio no SX5 com código Z2 :

Chave	Descrição
E	E-mail
L	Ligação
R	Reunião
V	Visita
W	WhatsApp

Campos virtuais: dados que aparecem sem ocupar espaço

Um **campo virtual** não é gravado no banco — ele é **calculado na hora** de mostrar. Isso evita duplicar informação. Configure no `X3_RELACAO` da SZ2:

Z2_CODIGO (código do contato SZ1):

```
POSICIONE("SZ1", 1, xFilial("SZ1") + M->Z2_CONTAT, "Z1_CODIGO")
```

Z2_ASSUNT (assunto do contato):

```
POSICIONE("SZ1", 1, xFilial("SZ1") + M->Z2_CONTAT, "Z1_ASSUNTO")
```

💡 `POSICIONE(alias, ordem, chave, campo)` “pula” até o registro cuja chave bate e devolve o valor de um campo. É a função que você mais vai usar para buscar dados de outra tabela.

9.3 A biblioteca de funções comuns (não repita código!)

Antes de escrever as rotinas, um passo que separa amadores de profissionais: **centralizar as funções que se repetem** numa **biblioteca**. Assim, se a regra mudar, você altera **em um só lugar**. Isso se chama princípio **DRY** (*Don't Repeat Yourself* — não se repita).

```

// =====
// STTIPLIB.PRW
// Biblioteca de funções comuns do módulo TIP
// =====

#include "protheus.ch"

// -----
// NomeCliente
// Retorna o nome do cliente a partir do código/loja
// -----
STATIC FUNCTION NomeCliente(cCodigo, cLoja)
    LOCAL cFilial := xFilial("SA1")
    LOCAL cNome

    IF Empty(cCodigo)
        RETURN ""
    ENDIF

    cNome := POSICIONE("SA1", 1, cFilial + cCodigo + cLoja, "A1_NOME")
RETURN AllTrim(cNome)

// -----
// ProxCodigoSZ1
// Retorna o próximo código disponível para SZ1
// -----
STATIC FUNCTION ProxCodigoSZ1()
RETURN GetSXENum("SZ1", "Z1_CODIGO")

// -----
// ProxSequenSZ2
// Retorna a próxima sequência para um contato na SZ2
// -----
STATIC FUNCTION ProxSequenSZ2(cContat)
    LOCAL nMax := 0

    dbSelectArea("SZ2")
    dbSetOrder(1)
    dbSeek(xFilial("SZ2") + cContat)

    WHILE !Eof() .AND. SZ2->Z2_FILIAL == xFilial("SZ2") .AND. SZ2->Z2_CONTAT == cContat
        nMax := Max(nMax, Val(SZ2->Z2_SEQUEN))
        dbSkip()
    ENDDO

RETURN StrZero(nMax + 1, 3)

// -----
// DescTipoInteracao
// Retorna a descrição do tipo de interação
// -----

```

```
STATIC FUNCTION DescTipoInteracao(cTipo)
  LOCAL cDesc := ""
  DO CASE
    CASE cTipo == "E" ; cDesc := "E-mail"
    CASE cTipo == "L" ; cDesc := "Ligação"
    CASE cTipo == "R" ; cDesc := "Reunião"
    CASE cTipo == "V" ; cDesc := "Visita"
    CASE cTipo == "W" ; cDesc := "WhatsApp"
  ENDCASE
  RETURN cDesc
```

Repare no padrão: cada função tem **um nome claro**, **um propósito único** e um **cabeçalho comentado**. Uma biblioteca bem escrita é lida como um índice: você bate o olho e sabe o que tem ali.

9.4 A rotina de Contatos (SZ1) — a tela “mãe”

Agora sim, a rotina principal. Além do CRUD padrão que você já conhece, ela traz **duas novidades profissionais: legendas coloridas** (o usuário vê o status pela cor) e um **botão de navegação** que abre as interações **daquele** contato.

```

// =====
// STTIP003.PRW
// Cadastro de Contatos (mBrowse)
// =====

#include "protheus.ch"

USER FUNCTION STTIP003()

    LOCAL cFiltro := ""
    LOCAL aColors

    PRIVATE cCadastro := "Contatos"
    PRIVATE aRotina := {;
        {"Pesquisar", "AxPesqui", 0, 1},;
        {"Visualizar", "AxVisual", 0, 2},;
        {"Incluir", "AxInclui", 0, 3},;
        {"Alterar", "AxAltera", 0, 4},;
        {"Excluir", "AxDeleta", 0, 5},;
        {"Interações", "U_STTIP004", 0, 6}; // ← abre a SZ2 filtrada!
    }

    // Legenda por idade do contato:
    // Verde: recente (até 7 dias)
    // Amarelo: antigo (8 a 30 dias)
    // Vermelho: muito antigo (> 30 dias)
    aColors := {;
        {"SZ1->Z1_DATA >= dDataBase - 7", "BR_GREEN"},;
        {"SZ1->Z1_DATA >= dDataBase - 30", "BR_YELLOW"},;
        {"T.", "BR_RED"};
    }

    dbSelectArea("SZ1")
    dbSetOrder(1)
    dbSeek(xFilial("SZ1"))

    mBrowse(1, 1, 22, 75, "SZ1", , , , , aColors, , , , .F., , , cFiltro)

    RETURN NIL

```


Entendendo as novidades

Elemento	O que faz
<code>aRotina</code>	Lista de opções do menu do browse. Cada item: <code>{"Título", "Função", 0, Operação}</code>
<code>{"Interações", "U_STTIP004", 0, 6}</code>	Opção customizada — chama outra rotina (nossa!)
<code>aColors</code>	Regras de cor: a primeira que der <code>.T.</code> pinta a linha
<code>mBrowse(...)</code>	Monta a grade (browse) com todos os recursos configurados

 Browse de Contatos com legendas coloridas

9.5 A rotina de Interações (SZ2) — a tela “filha”

Quando o usuário clica em “**Interações**” na tela de Contatos, o Protheus chama a rotina abaixo. O segredo está em **capturar o contato atual** e **filtrar** a SZ2 só por ele:

```
// =====
// STTIP004.PRW
// Interações de um Contato (SZ2)
// Chamada a partir da STTIP003 (botão "Interações")
// =====

#include "protheus.ch"

USER FUNCTION STTIP004()

    LOCAL cFiltro    := ""
    LOCAL cCodigSZ1 := SZ1->Z1_CODIGO    // código do registro atual da SZ1

    PRIVATE cCadastro := "Interações - Contato " + AllTrim(cCodigSZ1)
    PRIVATE aRotina    := {;
        {"Pesquisar", "AxPesqui", 0, 1},;
        {"Visualizar", "AxVisual", 0, 2},;
        {"Incluir", "AxInclui", 0, 3},;
        {"Alterar", "AxAltera", 0, 4},;
        {"Excluir", "AxDeleta", 0, 5};
    }

    // Filtro: mostrar apenas interações deste contato
    cFiltro := "Z2_CONTAT == '" + cCodigSZ1 + "'"

    dbSelectArea("SZ2")
    dbSetOrder(1)
    dbSeek(xFilial("SZ2") + cCodigSZ1)

    mBrowse(1, 1, 22, 75, "SZ2", , , , , , , , , , .F., , , cFiltro)

    RETURN NIL
```

Gatilhos: o sistema preenche por você

Um bom sistema **não pede ao usuário o que ele mesmo sabe**. Configure gatilhos (SX7) na SZ2 para preencher data, hora e usuário automaticamente:

Campo	Regra do gatilho	Fase
Z2_DATA	dDataBase	1 (só na inclusão)
Z2_HORA	IF(INCLUI, Time(), SZ2->Z2_HORA)	3 (ambos)
Z2_USUAR	cNomUsr (usuário logado)	1 (só na inclusão)

💡 `dDataBase` é a data-base do sistema; `Time()` é a hora atual; `cNomUsr` é o nome do usuário logado. Com esses três gatilhos, o usuário só digita o que interessa: **tipo e descrição**.

9.6 Validações cruzadas e integridade referencial

Duas tabelas relacionadas trazem uma responsabilidade nova: **garantir que a ligação entre elas faça sentido**. Isso se chama **integridade referencial** — não pode existir “filho órfão” nem apagar uma “mãe” que ainda tem filhos.

1. Toda interação precisa apontar para um contato que existe

No `X3_VALID` do campo `Z2_CONTAT` :

```
ExistCpo("SZ1", xFilial("SZ1") + M->Z2_CONTAT, 1)
```

`ExistCpo` verifica se a chave existe na tabela. Se não existir, o Protheus barra a digitação e avisa o usuário — impossível cadastrar interação para um contato inexistente.

2. Não deixe apagar um contato que tem interações

Se o usuário apagar um contato (SZ1) que ainda tem interações (SZ2), essas interações viram “órfãs”. Para impedir, crie uma validação de exclusão:

```
// =====  
// Impede excluir Contato (SZ1) com Interações (SZ2)  
// =====  
USER FUNCTION VALEXCSZ1()  
  
    IF ExistCpo("SZ2", xFilial("SZ2") + SZ1->Z1_CODIGO, 1)  
        MsgAlert("Contato possui interações! Exclua as interações primeiro.", "Atenção")  
        RETURN .F.    // cancela a exclusão  
    ENDIF  
  
RETURN .T.    // libera a exclusão
```

⚠️ **Validação de dados ≠ tratamento de erro.** Aqui você está validando uma **regra de negócio** (“não pode apagar mãe com filhos”) — para isso, `IF` + `RETURN .F.` basta. Guarde essa distinção: ela é a ponte para a próxima seção.

9.7 “E quando algo dá errado?” — Tratamento de erros

Você já validou o que o **usuário** digita. Mas e quando o problema **não é** o usuário?

Imagine a rotina de salvar um contato. Você validou tudo, chamou o comando de gravação... e nesse exato instante **o banco de dados fica inacessível**, a **rede cai** ou **outro usuário travou o registro**. Sem tratamento, o Protheus mostra uma mensagem técnica assustadora, os dados ficam **pela metade**, e o usuário liga furioso. Isso **não pode** acontecer num sistema profissional.

Sem tratamento	Com tratamento
Trava com mensagem técnica	Mensagem clara e amigável
Dados em estado inconsistente	Rollback automático da operação
Difícil de diagnosticar	Log técnico detalhado
Usuário perdido	Sistema continua funcionando

Os tipos de erro (e como tratar cada um)

Tipo	Quando ocorre	Trata com
Validação	Dados inválidos (CPF, campo vazio)	<code>IF / RETURN .F.</code>
Negócio	Regra da empresa violada (estoque, prazo)	<code>IF / RETURN .F.</code>
Runtime	Divisão por zero, acesso a NIL	<code>BEGIN SEQUENCE</code>
Banco de dados	Lock, conexão perdida	<code>BEGIN SEQUENCE</code>
Arquivo/Rede	Arquivo não encontrado, disco cheio	<code>BEGIN SEQUENCE</code>

Os dois primeiros você resolve com `IF` (como fez na seção 9.6). Os três últimos são **inesperados** — para eles existe o `BEGIN SEQUENCE`.

BEGIN SEQUENCE — o “try/catch” do ADVPL

```
BEGIN SEQUENCE
    // código que pode gerar erro
    // se ocorrer um erro, a execução SALTA para o RECOVER
RECOVER WITH oErro
    // executado só em caso de erro
    // oErro traz os detalhes do que aconteceu
END SEQUENCE

// o programa continua daqui, com ou sem erro
```

Um exemplo simples, para ver o mecanismo funcionando:

```
LOCAL nA := 10
LOCAL nB := 0
LOCAL nResultado

BEGIN SEQUENCE
    nResultado := nA / nB    // divisão por zero!
    QOut("Resultado: " + Str(nResultado))
RECOVER WITH oErro
    QOut("Erro capturado!")
    QOut("Descrição: " + oErro:Description)
END SEQUENCE

QOut("Programa continua normalmente.")
```

Saída:

```
Erro capturado!
Descrição: Zero divisor
Programa continua normalmente.
```

Sem o `BEGIN SEQUENCE`, aquela divisão por zero **derrubaria** o programa inteiro.

O objeto de erro (oErro)

Ao capturar com `RECOVER WITH oErro`, você recebe um objeto cheio de informações úteis:

Propriedade	Descrição
<code>oErro:Description</code>	Descrição do erro
<code>oErro:Operation</code>	Operação que causou o erro
<code>oErro:SubSystem</code>	Subsistema (DBFNTX, NETSIO...)
<code>oErro:SubCode</code>	Código do erro
<code>oErro:ProcName</code>	Função onde o erro ocorreu
<code>oErro:ProcLine</code>	Linha onde o erro ocorreu

 Fluxo do BEGIN SEQUENCE capturando um erro

9.8 Juntando tudo: a rotina de salvar profissional

Aqui a fusão fica completa. Uma gravação profissional combina **quatro camadas**: validação de negócio (`IF`), **transação** (para poder desfazer), `BEGIN SEQUENCE` (para capturar o inesperado) e **log** (para diagnosticar depois).

```

// =====
// Gravação segura de um Contato (SZ1)
// =====
USER FUNCTION STTIP003SALVAR()

    LOCAL lok := .T.
    LOCAL oErro

    BeginTran()    // 1. inicia a transação (tudo ou nada)

    BEGIN SEQUENCE

        // 2. validação de negócio → IF/Break
        IF Empty(M->Z1_CLIENTE)
            MsgAlert("Cliente é obrigatório!", "Atenção")
            lok := .F.
            Break()    // força a saída para o RECOVER (sem erro real)
        ENDIF

        // 3. operação no banco, protegida por lock
        dbSelectArea("SZ1")
        IF INCLUI
            RecLock("SZ1", .T.)    // .T. = novo registro
        ELSE
            RecLock("SZ1", .F.)    // .F. = altera o atual
        ENDIF

        SZ1->Z1_CODIGO    := M->Z1_CODIGO
        SZ1->Z1_CLIENTE    := M->Z1_CLIENTE
        SZ1->Z1_ASSUNTO    := M->Z1_ASSUNTO
        MsUnLock()        // salva e libera o lock

    RECOVER WITH oErro
        // 4. deu erro inesperado: desfaz tudo, avisa e loga
        lok := .F.
        RollBackTran()
        MsgStop("Erro ao salvar: " + oErro:Description, "Erro")
        U_GRAVARLOG("STTIP003SALVAR", oErro)
        RETURN lok
    END SEQUENCE

    IF lok
        CommitTran()    // 5. tudo certo: confirma a transação
    ENDIF

    RETURN lok

```

Por que transação (`BeginTran`) e lock (`RecLock`)?

- `BeginTran` / `CommitTran` / `RollBackTran` garantem o princípio “**tudo ou nada**”: se algo falhar no meio, o `RollBackTran()` desfaz **todas** as gravações — nada fica pela metade.
- `RecLock` / `MsUnLock` garantem que **dois usuários não editem o mesmo registro ao mesmo tempo**. `RecLock("SZ1", .T.)` cria um registro novo; `RecLock("SZ1", .F.)` trava o atual para alteração; `MsUnLock()` salva e libera.

A função de log

O log é o que permite **diagnosticar em produção** um erro que você não viu acontecer:

```
// =====  
// STTIPLIB.PRW – adicione esta função à sua lib  
// =====  
USER FUNCTION GRAVARLOG(cFuncao, oErro)  
  
    LOCAL cArqLog := "\\logs\advpl_" + DToS(Date()) + ".log"  
    LOCAL nHandle  
    LOCAL cLinha  
  
    cLinha := DToS(Date()) + " " + Time() + " | "  
    cLinha += cFuncao + " | "  
    cLinha += cNomUsr + " | "  
    cLinha += "Empresa: " + cEmpAnt + " Filial: " + cFilAnt + " | "  
  
    IF oErro != NIL  
        cLinha += "ERRO: " + oErro:Description + " | "  
        cLinha += "Func: " + oErro:ProcName + ":" + cValToChar(oErro:ProcLine) + " | "  
        cLinha += "SubSist: " + oErro:SubSystem + " | "  
        cLinha += "Oper: " + oErro:Operation  
    ENDIF  
  
    nHandle := FOpen(cArqLog, FO_READWRITE + FO_SHARED)  
    IF nHandle < 0  
        nHandle := FCreate(cArqLog)  
    ENDIF  
    FSeek(nHandle, 0, FS_END) // vai para o fim do arquivo  
    FWrite(nHandle, cLinha + CRLF)  
    FClose(nHandle)  
  
    RETURN NIL
```

ERRORBLOCK: uma rede de segurança global

E se um erro escapar de **todos** os `BEGIN SEQUENCE` ? O `ErrorBlock()` instala um **tratador global** — a última rede de proteção do módulo:


```

USER FUNCTION MEUMENU()

    LOCAL bOldError := ErrorBlock({|oE| U_MEUTRATADOR(oE)})    // instala
    U_STTIP003()    // todo o módulo roda "protegido"

    ErrorBlock(bOldError)    // restaura o handler original ao sair

RETURN NIL

USER FUNCTION MEUTRATADOR(oErro)
    MsgStop("Ocorreu um erro inesperado. A equipe de TI foi notificada.", "Erro")
    U_GRAVARLOG("ERRO GLOBAL", oErro)
RETURN Break(oErro)    // propaga o erro após tratar

```

Boas práticas (o resumo que vale ouro)

1. **IF** para o esperado, **BEGIN SEQUENCE** para o inesperado. Validação de dados é **IF** ; erro de banco/rede é **BEGIN SEQUENCE** .
2. Toda operação em transação faz rollback no erro. **RollBackTran()** no **RECOVER** , sempre.
3. Mensagem amigável para o usuário, detalhe técnico no log. O usuário não precisa ver **oErro:SubCode** — o desenvolvedor sim.
4. Nunca engula um erro em silêncio. Um **RECOVER** vazio é uma bomba-relógio: no mínimo, grave o log.

9.9 O menu e a montagem final

Para o usuário chegar às rotinas, configure o menu no **SIGACFG > Menu do Sistema** (módulo SIGACOM, por exemplo):

```

SIGACOM
├─ Cadastros
│   └─ Contatos          → USER FUNCTION STTIP003
│   └─ Interações (todas) → USER FUNCTION STTIP004B

```

E, nas rotinas, aponte a biblioteca para reaproveitar as funções:

```

// no appserver.ini ou via SET PROCEDURE
SET PROCEDURE TO STTIPLIB

```

Estrutura final do projeto:

```
STTIPLIB.PRW    ← biblioteca de funções comuns (+ GRAVARLOG)
STTIP003.PRW    ← Contatos (SZ1) – mBrowse com legendas + botão Interações
STTIP004.PRW    ← Interações (SZ2) – filtrada pelo contato selecionado
STTIP004B.PRW   ← Interações (SZ2) – listagem geral, para o menu
```

👉 Mão na massa (em aula)

Vamos codar juntos a peça mais importante do sistema: **a gravação à prova de falhas**.

A partir do esqueleto abaixo, complete a `U_STTIP003SALVAR()` para que ela: 1. Valide que `Z1_CLIENTE` e `Z1_ASSUNTO` estão preenchidos (senão, avisa e sai). 2. Grave dentro de `BeginTran()` + `BEGIN SEQUENCE`. 3. Em caso de erro, faça `RollBackTran()`, mostre mensagem amigável e chame `U_GRAVARLOG()`. 4. Só chame `CommitTran()` se tudo deu certo.

```
USER FUNCTION STTIP003SALVAR()
    LOCAL lOk := .T.
    LOCAL oErro

    BeginTran()
    BEGIN SEQUENCE
        // 1) valide os campos obrigatórios aqui (use Break() se faltar algo)

        // 2) grave com RecLock / MsUnLock aqui

    RECOVER WITH oErro
        // 3) rollback + mensagem + log aqui
    END SEQUENCE

    // 4) commit se lOk aqui
    RETURN lOk
```

No fim, cada um roda a rotina e provoca um erro de propósito (ex.: cliente vazio) para ver a mensagem amigável aparecer — **sem** o sistema travar. 🎯

Referência rápida do Módulo 9

```
// --- Navegação entre tabelas ---
POSICIONE("SZ1", 1, xFilial("SZ1") + cChave, "Z1_NOME") // busca campo de outra
tabela
ExistCpo("SZ1", xFilial("SZ1") + cChave, 1)           // .T. se a chave existe

// --- Browse com opção customizada e legendas ---
PRIVATE aRotina := { {"Interações", "U_STTIP004", 0, 6} }
aColors := { {"SZ1->Z1_DATA >= dDataBase - 7", "BR_GREEN"}, {".T.", "BR_RED"} }
mBrowse(1, 1, 22, 75, "SZ1", , , , , aColors, , , , .F., , , cFiltro)

// --- Gravação segura (transação + lock + erro) ---
BeginTran()
BEGIN SEQUENCE
    RecLock("SZ1", .T.) // .T. inclui .F. altera
    SZ1->Z1_CODIGO := M->Z1_CODIGO
    MsUnLock()       // salva e libera o lock
    CommitTran()     // confirma
RECOVER WITH oErro
    RollBackTran()   // desfaz tudo
    MsgStop("Erro: " + oErro:Description, "Erro")
    U_GRAVARLOG("Salvar", oErro)
END SEQUENCE

// --- Tratamento global ---
bOld := ErrorBlock({|oE| U_MEUTRATADOR(oE)}) // instala . restaure com
ErrorBlock(bOld)
```

Propriedade do **oErro**

O que traz

:Description

O que deu errado

:ProcName / **:ProcLine**

Onde ocorreu (função / linha)

:Operation / **:SubSystem**

Operação e subsistema que falharam



Exercícios

Os exercícios desta aula estão em **exercicios.md** (nesta mesma pasta). Você vai montar o projeto completo (tabelas, lib, rotinas, validações) e blindar a gravação com tratamento de erros. **Prazo sugerido:** até 31/07.



Para a próxima aula

Módulo 10 — Orientação a Objetos (OOP), em 30/07. Depois, em 31/07, teremos **TCC + encerramento**. 🏁 Você vai conhecer o paradigma de **objetos** em ADVPL (classes, atributos, métodos, herança) e — o momento mais esperado — receber as orientações de **como entregar o seu TCC**.



Antes da Aula 10: deixe o seu projeto de Contatos/Interações compilando. O TCC é uma evolução direta dele (duas tabelas relacionadas, lib, validações e tratamento de erros) — quem caprichou aqui vai começar o TCC na frente.